

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

APPENDIX E

ONE-TO-ONE | SERVERPOD

 Copy as Markdown

One-to-one

One-to-one (1:1) relationships represent a unique association between two entities. At most one model can be connected on either side of the relation. This means we have to set a **unique index** on the foreign key in the database. Without the unique index the relation would be considered a one-to-many (1:n) relation.

Defining the relationship

In the following examples we show how to configure a 1:1 relationship between `User` and `Address`.

With an id field

In the most simple case, all we have to do is add an `id` field on one of the models.

```
# address.spy.yaml
class: Address
table: address
fields:
  street: String

# user.spy.yaml
class: User
table: user
fields:
  # The foreign key field.
  addressId: int, relation(parent=address)
indexes:
  user_address_unique_idx:
    fields: addressId
    unique: true
```

In the example, the `relation` keyword annotates the `addressId` field to hold the foreign key. The type must match the parent table's id type: `int` here, or `UuidValue` when the parent model uses



Ask AI

The relation keyword needs to specify the `parent` parameter. The `parent` parameter defines which table the relation is towards, in this case the `Address` table.

The `addressId` is **required** in this example because the field is not nullable. That means that each `User` must have a related `Address`. If you want to make the relation optional, make the datatype nullable (`int?`).

When fetching a `User` from the database the `addressId` field will automatically be populated with the related `Address` object `id`.

With an object

While the previous example highlights manual handling of data, there's an alternative approach that simplifies data access using automated handling. By directly specifying the `Address` type in the `User` class, Serverpod can automatically handle the relation for you.

```
# address.spy.yaml
class: Address
table: address
fields:
  street: String

# user.spy.yaml
class: User
table: user
fields:
  # The object relation field.
  address: Address?, relation
indexes:
  user_address_unique_idx:
    fields: addressId
    unique: true
```

In this example, we define an object relation field by annotating the `address` field with the `relation` keyword where the type is another model, `Address?`.

Serverpod then automatically generates a foreign key field (as seen in the last example) named `addressId` in the `User` class. This auto-generated field is non-nullable by default and is by default always named from the object relation field with the suffix `Id`.

The object field, in this case `address`, must always be nullable (as indicated by `Address?`).

An object relation field gives a big advantage when fetching data. Utilizing [relational queries](#) enables filtering based on relation attributes or optionally including the related data in the result.

No `parent` keyword is needed here because the relational table is inferred from the type on the field.

Optional relation

```
# user.spy.yaml
class: User
table: user
fields:
  address: Address?, relation(optional)
indexes:
  user_address_unique_idx:
    fields: addressId
    unique: true
```

With the introduction of the `optional` keyword in the relation, the automatically generated `addressId` field becomes nullable. This means that the `addressId` can either hold a foreign key to the related `address` table or be set to null, indicating no associated address.

Custom foreign key field

The `field` parameter names the foreign key field used by an object relation.

```
# user.spy.yaml
class: User
table: user
fields:
  address: Address?, relation(field=customIdField)
indexes:
  user_address_unique_idx:
    fields: customIdField
    unique: true
```

In this example, `customIdField` holds the foreign key instead of the default auto-generated name `addressId`. The field does not have to be declared in `fields`. Serverpod generates it with the id type of the related model, just like it does for a relation without the `field` parameter, and the `optional` keyword makes it nullable:

```
fields:
  address: Address?, relation(optional, field=customIdField)
```

Declare the field yourself when you need control over it, for example to give it a `column name override` or a `scope`. A declared field must be nullable if the relation is `optional`:

```
# user.spy.yaml
class: User
table: user
fields:
  customIdField: int?, column=fk_user_address_id
  address: Address?, relation(optional, field=customIdField)
indexes:
  user_address_unique_idx:
```

```
fields: customIdField
unique: true
```

Generated SQL

The following code block shows how to set up the same relation with raw SQL. Serverpod will generate this code behind the scenes.

```
CREATE TABLE "address" (
  "id" bigserial PRIMARY KEY,
  "street" text NOT NULL
);

CREATE TABLE "user" (
  "id" bigserial PRIMARY KEY,
  "addressId" bigint NOT NULL
);

CREATE UNIQUE INDEX "user_address_unique_idx" ON "user" USING btree ("addressId");

ALTER TABLE ONLY "user"
  ADD CONSTRAINT "user_fk_0"
  FOREIGN KEY("addressId")
  REFERENCES "address"("id")
  ON DELETE NO ACTION
  ON UPDATE NO ACTION;
```

The `ON DELETE` and `ON UPDATE` clauses come from the relation's referential actions, which default to `NO ACTION`. See [Referential actions](#) to change them.

Independent relations defined on both sides

You are able to define as many independent relations as you wish on each side of the relation. This is useful when you want to have multiple relations between two entities.

```
# user.spy.yaml
class: User
table: user
fields:
  friendsAddress: Address?, relation
indexes:
  user_address_unique_idx:
    fields: friendsAddressId
    unique: true

# address.spy.yaml
class: Address
table: address
fields:
```

```

street: String
resident: User?, relation
indexes:
  address_user_unique_idx:
    fields: residentId
    unique: true

```

Both relations operate independently of each other, resulting in two distinct relationships with their respective unique indexes.

Bidirectional relations

If access to the same relation is desired from both sides, a bidirectional relation can be defined.

```

# user.spy.yaml
class: User
table: user
fields:
  address: Address?, relation(name=user_address, field=addressId)
indexes:
  user_address_unique_idx:
    fields: addressId
    unique: true

# address.spy.yaml
class: Address
table: address
fields:
  street: String
  user: User?, relation(name=user_address)

```

The example illustrates a 1:1 relationship between User and Address where both sides of the relationship are explicitly specified.

Using the `name` parameter, we define a shared name for the relationship. It serves as the bridge connecting the `address` field in the User class to the `user` field in the Address class. Meaning that the same `User` referencing an `Address` is accessible from the `Address` as well.

Without specifying the `name` parameter, you'd end up with two unrelated relationships.

When the relationship is defined on both sides, you must mark which side holds the foreign key, because Serverpod cannot decide that for you. Either name the foreign key field with `field`, as above, or add the `fk` flag to keep the generated `addressId` name:

```

# user.spy.yaml
class: User
table: user
fields:

```

```
address: Address?, relation(name=user_address, fk)
indexes:
  user_address_unique_idx:
    fields: addressId
    unique: true
```

Only one side of a relation can hold the key. As with any [custom foreign key field](#), `addressId` itself is generated and only needs to be declared if you want control over it.

In a relationship where there is an object on both sides a unique index is always **required** on the foreign key field.